

Android (Kotlin) + Firebase (Messaging)

1. Crie um projeto e altere o SDK para 37.
2. Configure o Firebase (como ensinado em aula).
3. No **build.gradle.kts (módulo:app)**, adicione as dependências:

```
implementation("com.google.firebase:firebase-messaging")  
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")  
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.11.0")
```

4. No AndroidManifest.xml:

```
<!-- Necessário para acessar a internet e consumir a API -->  
<uses-permission android:name="android.permission.INTERNET" />  
  
-----  
<activity  
    android:name=".MainActivity"  
    android:exported="true"  
    android:label="@string/app_name"  
    android:theme="@style/Theme.ModuloConsumoAPI4">  
    <intent-filter> <action android:name="android.intent.action.MAIN" />  
        <category android:name="android.intent.category.LAUNCHER" />  
    </intent-filter>  
</activity>
```

5. Vamos criar a camada **model**, ela ficará dentro da camada **data**. Sua estrutura será composta por:

```
data class PushNotification(  
    val title: String,  
    val message: String  
)
```

6. Agora vamos criar a camada **repository**, ela ficará dentro da camada **data**. Sua estrutura será composta por:

```
import com.google.firebase.messaging.FirebaseMessaging  
import kotlinx.coroutines.tasks.await  
  
class FirebaseMessagingRepository {  
  
    suspend fun getToken(): String {  
  
        return FirebaseMessaging  
            .getInstance()  
            .getToken()  
            .await()
```

```
}  
}
```

7. Vamos criar a camada **notification**, dentro da camada **data**. Crie dois arquivos, com as seguintes estruturas:

```
import android.app.NotificationChannel  
import android.app.NotificationManager  
import android.content.Context  
import android.os.Build  
import androidx.annotation.RequiresApi  
import androidx.core.app.NotificationCompat  
import br.com.treinamento.projetonotificacao.R
```

```
object NotificationHelper {
```

```
    private const val CHANNEL_ID = "firebase_channel"
```

```
    @RequiresApi(Build.VERSION_CODES.O)
```

```
    fun showNotification(  
        context: Context,  
        title: String,  
        message: String  
    ) {
```

```
        val manager =
```

```
            context.getSystemService(  
                Context.NOTIFICATION_SERVICE
```

```
            ) as NotificationManager
```

```
        val channel =
```

```
            NotificationChannel(  
                CHANNEL_ID,  
                "firebase_channel",  
                NotificationManager.IMPORTANCE_DEFAULT
```

```

        CHANNEL_ID,
        "Firebase Notifications",
        NotificationManager.IMPORTANCE_HIGH
    )
    manager.createNotificationChannel(channel)
    val notification =
        NotificationCompat.Builder(
            context,
            CHANNEL_ID
        )
        .setSmallIcon(
            R.drawable.ic_launcher_foreground
        )
        .setContentTitle(title)
        .setContentText(message)
        .setPriority(
            NotificationCompat.PRIORITY_HIGH
        )
        .build()

    manager.notify(
        1,
        notification
    )
}
}

```

```
import android.os.Build
import androidx.annotation.RequiresApi
import com.google.firebase.messaging.FirebaseMessagingService
import com.google.firebase.messaging.RemoteMessage

class MyFirebaseMessagingService : FirebaseMessagingService() {

    @RequiresApi(Build.VERSION_CODES.O)
    override fun onMessageReceived(
        remoteMessage: RemoteMessage
    ) {

        val title =
            remoteMessage.notification?.title
            ?: "Nova notificação"

        val message =
            remoteMessage.notification?.body
            ?: "Você recebeu uma mensagem"

        NotificationHelper.showNotification(
            context = applicationContext,
            title = title,
            message = message
        )
    }
}
```

```

override fun onNewToken(token: String) {
    super.onNewToken(token)

    // Esse método é chamado quando o Firebase
    // gera um novo token para o dispositivo.

    println("Novo token FCM: $token")
}
}

```

8. Agora podemos criar a camada **viewmodel**, e implementar essa estrutura:

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import
br.com.treinamento.projetonotificacao.data.repository.FirebaseMessagingRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

class NotificationViewModel : ViewModel() {

    private val repository = FirebaseMessagingRepository()

    private val _token = MutableStateFlow("")

    val token: StateFlow<String> =

```

`_token`

```
private val _notification =  
    MutableStateFlow<PushNotification?>(null)  
  
val notification: StateFlow<PushNotification?> =  
    _notification
```

```
fun loadToken() {  
  
    viewModelScope.launch {  
  
        try {  
  
            _token.value =  
                repository.getToken()  
  
        } catch (e: Exception) {  
  
            _token.value =  
                "Erro: ${e.message}"  
  
            e.printStackTrace()  
        }  
    }  
}
```

```
fun onNotificationReceived(  
    notification: PushNotification  
) {
```

```

        _notification.value =
            notification
    }
}

```

9. Vamos para a última camada, crie o pacote **view**:

```

@Composable
fun NotificationScreen(viewModel: NotificationViewModel = viewModel()) {

    // Obter token
    val token by viewModel.token.collectAsState()

    // Notificação
    val notification by viewModel.notification.collectAsState()

    // Estrutura
    Column(
        modifier =
            Modifier
                .padding(20.dp)
    ) {

        Button(
            onClick = {
                viewModel.loadToken()
            }
        ) {
            Text("Gerar Token")
        }
    }
}

```



```

        Spacer(Modifier.height(20.dp))

        Text(text = "Token:")

        Text(text = token)

        Spacer(Modifier.height(20.dp))

        notification?.let {
            Text(text = "Mensagem: ${it.message}")
        }

    }
}

```

10. No arquivo **MainActivity**, precisamos fazer o seguinte:

a. Adicionar antes do override function:

// Registra o gerenciador de resultado que escuta e processa a resposta do usuário sobre a permissão.

```
private val notificationPermissionLauncher =
```

```
registerForActivityResult(ActivityResultContracts.RequestPermission()) { isGranted ->
```

```
    if (isGranted) {
```

```
        println("Permissão de notificação concedida")
```

```
    } else {
```

```
        println("Permissão de notificação negada")
```

```
    }
```

```
}
```

// Função que inicia o fluxo visual pedindo a permissão ao usuário.

```
private fun requestNotificationPermission() {
```

```
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
```

```
        notificationPermissionLauncher.launch(
```

```
        Manifest.permission.POST_NOTIFICATIONS
    )
}
}
```

b. Adicione após o super: **requestNotificationPermission()**

c. Chame a tela principal: **NotificationScreen()**

11. Teste a aplicação.